AIAC
ACADÉMIE INTERNATIONALE MOHAMMED VI DE L'AVIATION CIVILE

Prediction des retards de vols

Realisé par:

Khaoula El Kihal
Hajar Harbouli
Assiya Ouaradi

Hajar El Maimouni
Nassira Ouzzine
Mariam EL Hajoui

Annee universitaire : 2025 / 2026

Introduction

Le transport aérien est un secteur fortement impacté par les retards de vols, causés par plusieurs facteurs.

L’objectif principal est de développer un système prédictif capable d’anticiper les retards de vols en les classant en trois catégories :

- Vol à l’heure
- Retard supérieur à 15 minutes
- Retard supérieur à 30 minutes

Partie 1:
Data Preprocessing & Feature Engineering

1. Exploration de la data

- Taille de dataset significative :

NB: On a reduit la taille de la dataset pour creer un prototype

```
Flights shape   : (269102, 31)
Airlines shape  : (14, 2)
Airports shape  : (322, 7)
```

dataset suffisamment volumineux et diversifié pour construire et évaluer des modèles de Machine Learning fiables.

1. Exploration de la data

- **fligths.csv** fichier avec colonnes riches et diversifies:

-Variables temporelles:
- Year
- Month
- Day
- Day of week

-Informations sur le vol:
- Airline
- Flight number
- Aircraft ID

-Informations sur les aéroports:
- Origin airport
- Destination airport
- Distance

-Informations horaires:
- Scheduled departure
- Actual departure
- Arrival time

1. Exploration de la data

- **fligths.csv** fichier colonnes riches et diversifies:

  - Variables de retard:
    - Departure delay
    - Arrival delay
    - Weather delay
    - Airline delay

N.B: notre target sera Delay class cree a base de Arrival delay lors de la phase de feature engineering.

1. Exploration de la data

- **airports.csv** contient:
  IATA_CODE
  AIRPORT
  CITY
  STATE
  COUNTRY
  LATITUDE
  LONGITUDE

➡ Fournit des informations géographiques et de localisation sur les aéroports utilisés dans les vols.

- **airplanes.csv** contient: IATA_CODE et AIRLINE

➡ Permet d'identifier et d'associer chaque vol à sa compagnie aérienne

1. Exploration de la data

- **Pourcentage des valeurs manquantes par colonne:**

CANCELLATION_REASON	97.946132
LATE_AIRCRAFT_DELAY	74.490342
WEATHER_DELAY	74.490342
AIRLINE_DELAY	74.490342
AIR_SYSTEM_DELAY	74.490342
SECURITY_DELAY	74.490342
ARRIVAL_DELAY	2.284636

1. Exploration de la data

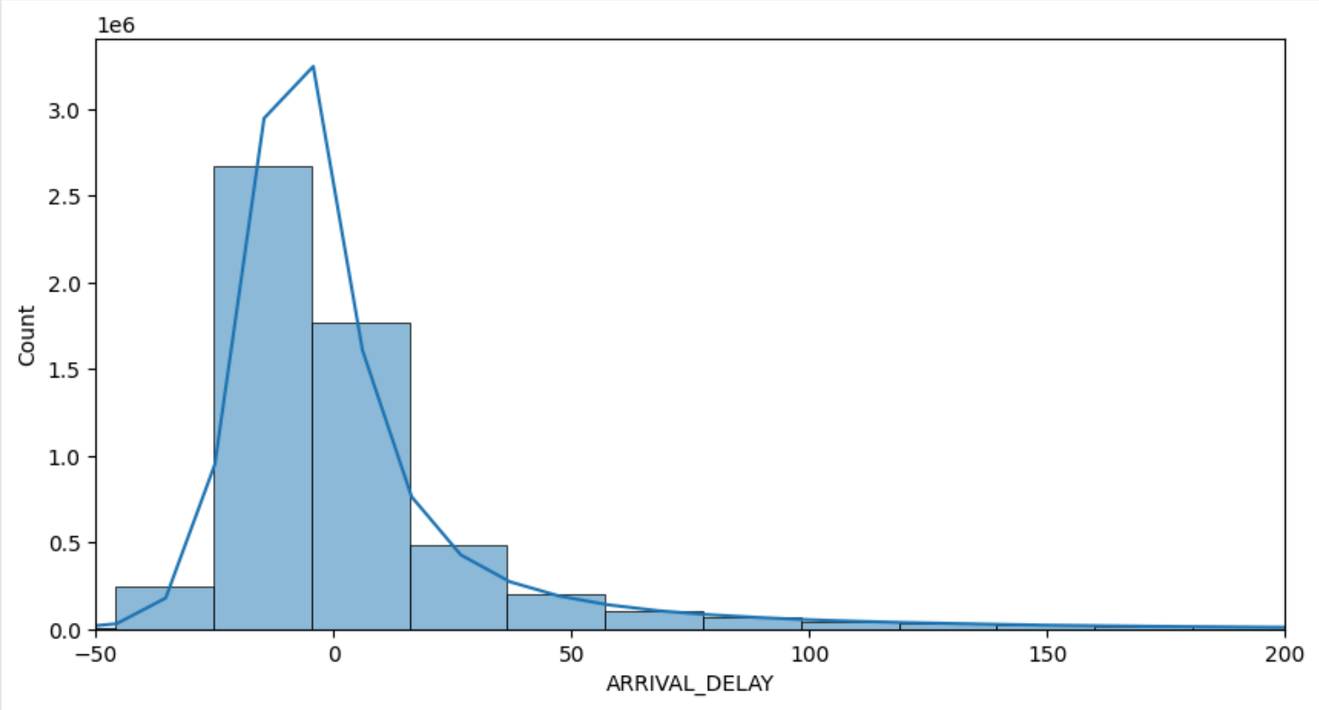
- **Verification des doublons :**

```
flights.duplicated().sum()
```

```
np.int64(0)
```

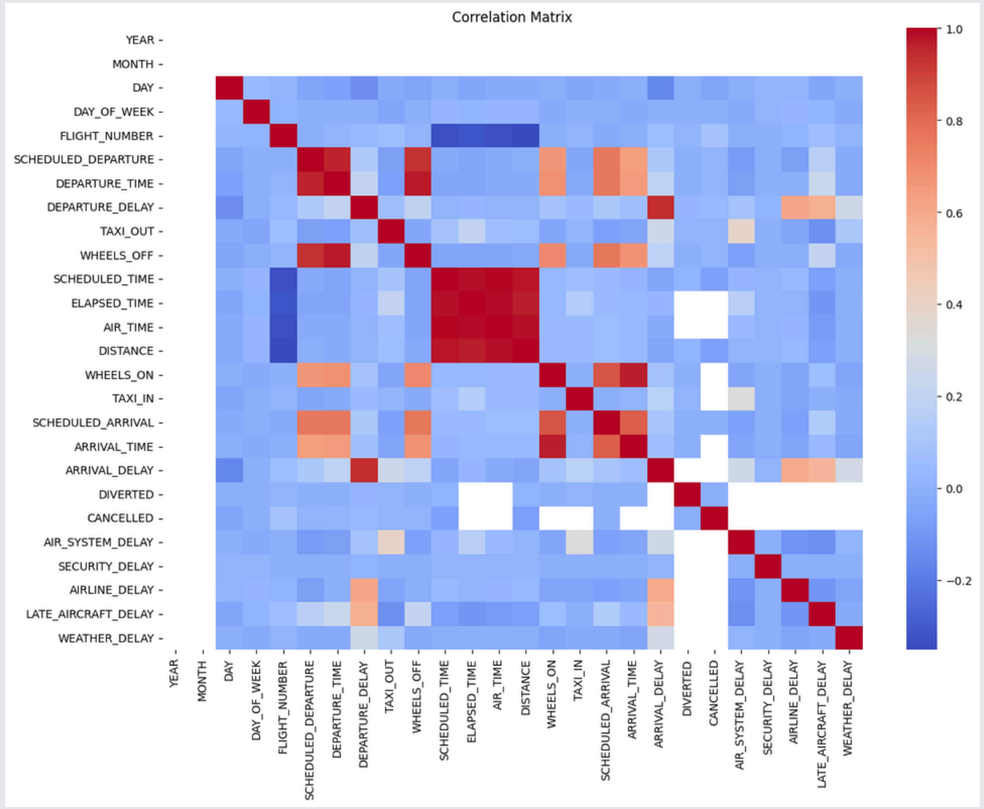
1. Exploration de la data

- **Distribution de la target ARIVAL DELAY:**



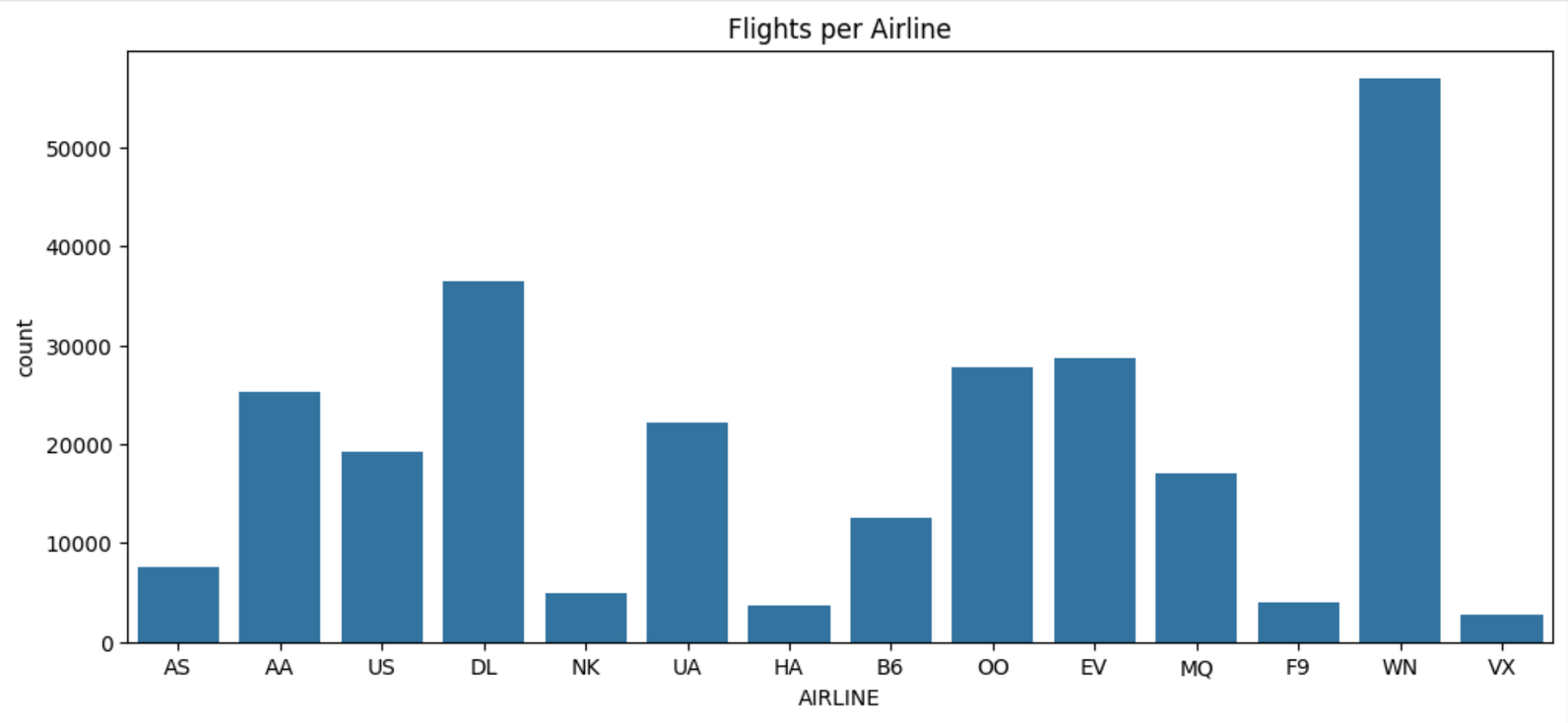
1. Exploration de la data

- **Correlation entre les variables numeriques:**



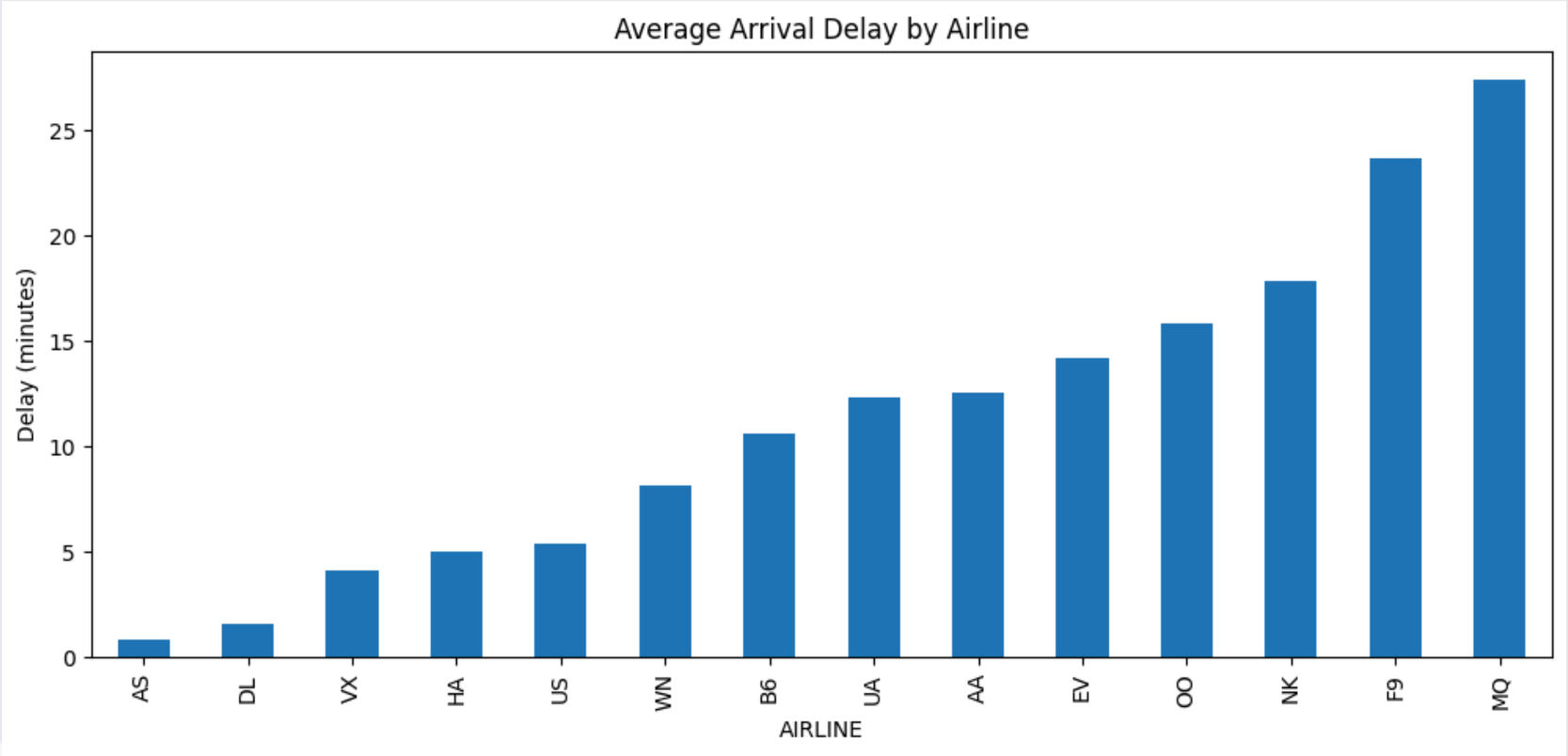
1. Exploration de la data

- **Analyse des compagnies aériennes:**



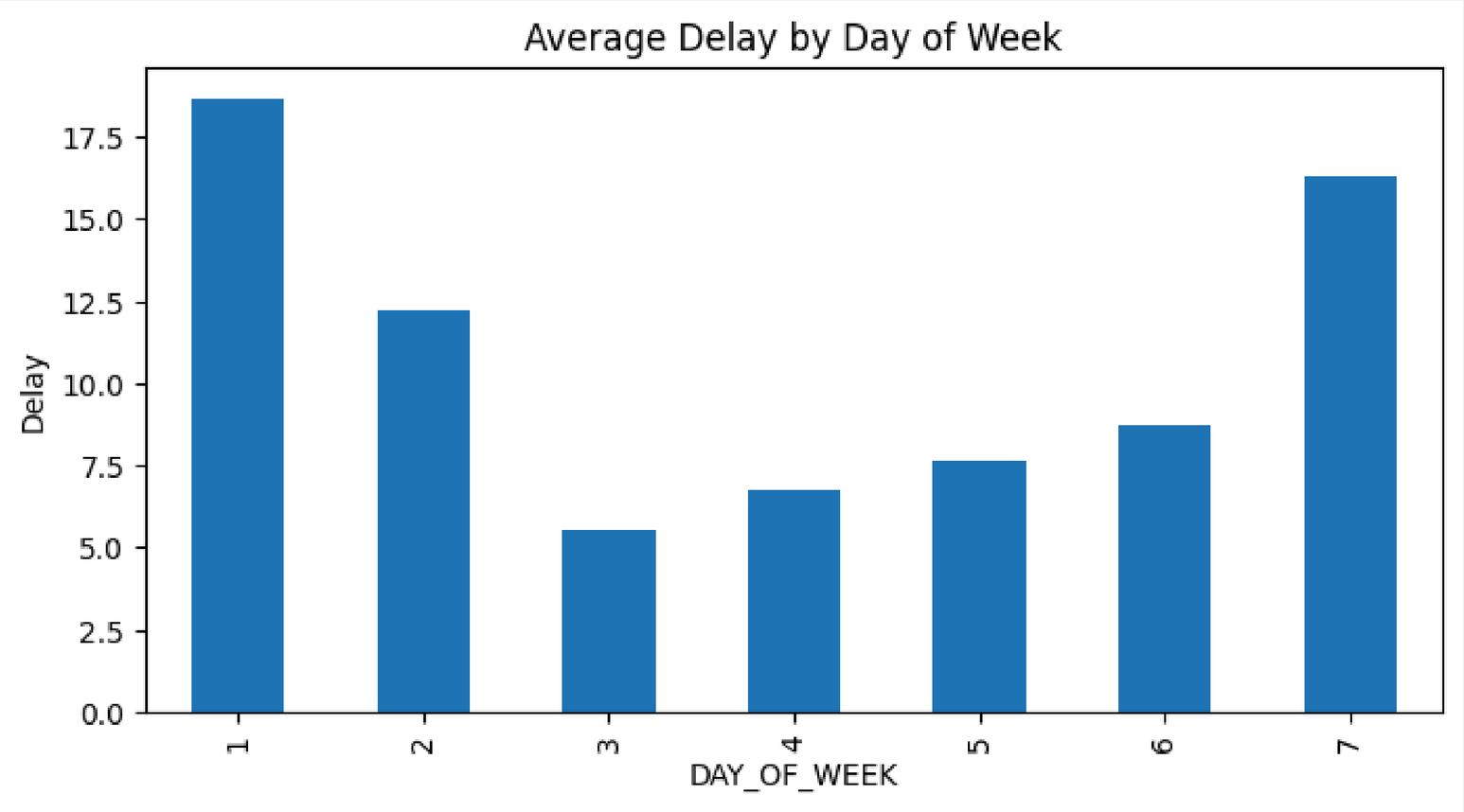
1. Exploration de la data

- **Retard moyen par compagnie:**



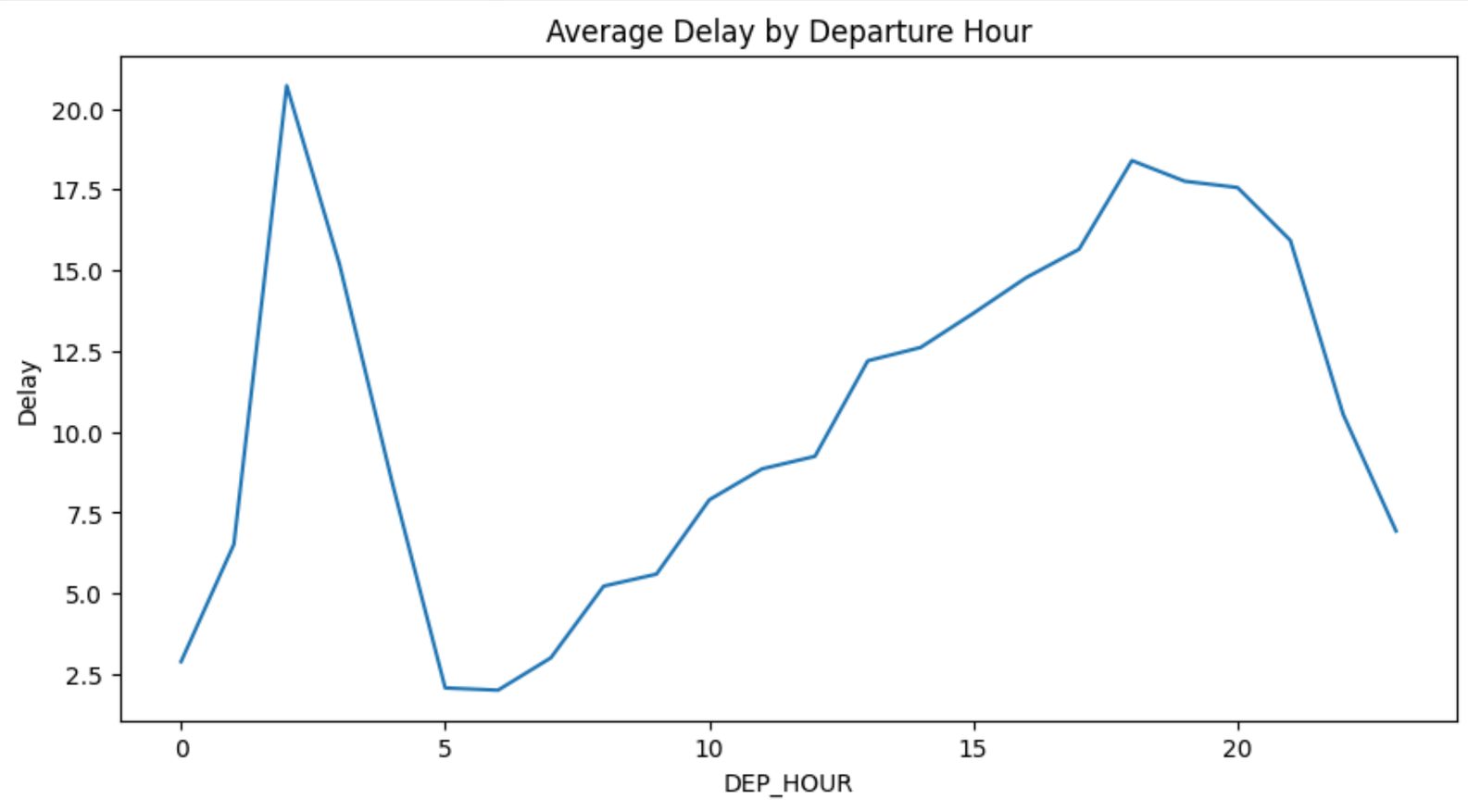
1. Exploration de la data

- **Retard selon jour semaine:**



1. Exploration de la data

- **Retard selon heure départ:**



2. Data cleaning

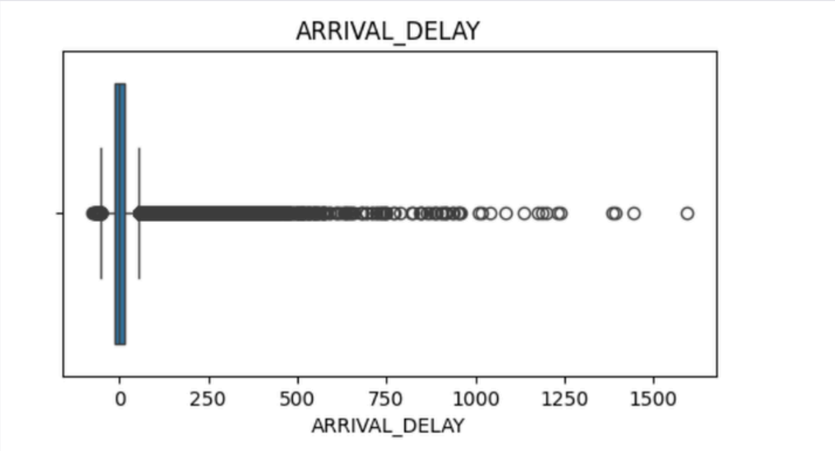
- **Suppression des colonnes innutiles et qui representes des fuites de donnees:**

```
cols_to_drop = [
    "CANCELLATION_REASON",
    "AIR_SYSTEM_DELAY",
    "SECURITY_DELAY",
    "AIRLINE_DELAY",
    "LATE_AIRCRAFT_DELAY",
    "WEATHER_DELAY",
    "WHEELS_OFF",
    "WHEELS_ON",
    "TAXI_IN",
    "TAXI_OUT",
    "DEPARTURE_TIME",
    "ARRIVAL_TIME",
    "ELAPSED_TIME",
    "AIR_TIME"
]
```

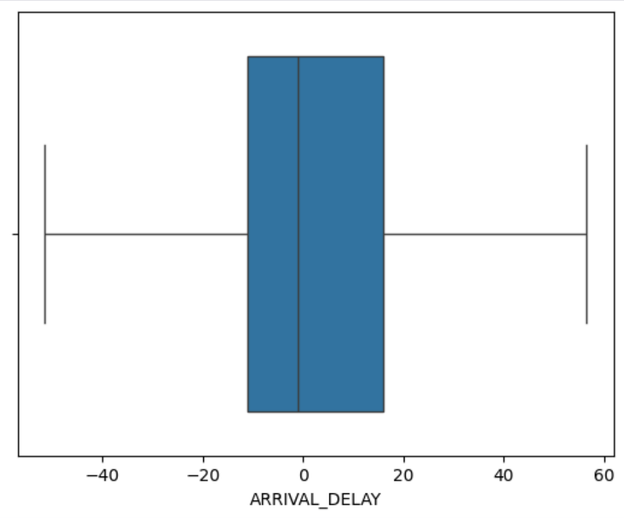
2. Data cleaning

- **Traitement des outliers avec la méthode IQR (InterQuartile Range) :**

Avant traitement:



Apres traitement:



3. Feature engineering:

- **Creation de la target de classification: DELAY_CLASS**

  ➡ Transformation de la variable cible en classification multi-classes

  0 : vol à l'heure ou léger retard

  1 : retard modéré

  2 : retard important

  Cela permet d'adapter le problème aux modèles de classification.

3. Feature engineering:

- **Autre features crees:**

  ➡ **Temporelle:** DEP_HOUR, ARR_HOUR

  ➡ **Catégorielle:** DAY_PERIOD, SEASON, WEATHER_RISK

  ➡ **Numérique:** AIRPORT_TRAFFIC

  ➡ **Binaire:** WEEKEND

4. Encodage:

➡ **One-Hot Encoding:** AIRLINE, DAY_PERIOD, SEASON, WER_RISK

➡ **Frequency Encoding:** ORIGIN_AIRPORT, DESTINATION_AIRPORT,

- **Taille dtaset final apres encodage:**

```
df_fe.shape
```

```
(262954, 33)
```

Partie 2:
Machine Learning &Optimisation

1. Objectif & Contexte

- **- Données reçues : 262 793 vols américains (2015)**
- **- 32 features disponibles après preprocessing**

- **Entraîner des modèles ML capables de prédire si un vol sera en retard ou non**

- **- 3 classes à prédire :**
  → Classe 0 : À l'heure
  → Classe 1 : Retard de +15 minutes
  → Classe 2 : Retard de +30 minutes

2. Préparation des données

- - Split : 80% Train (210 234 vols) / 20% Test (52 559 vols)
-
- - Normalisation : StandardScaler
- → Moyenne = 0 / Écart-type = 1
- → Appliqué uniquement sur le train (éviter data leakage)
-
- - Déséquilibre des classes :
- → Classe 0 (À l'heure) : 74.8%
- → Classe 1 (+15 min) : 9.1% ⚠️ sous-représentée
- → Classe 2 (+30 min) : 16.1%

→ Solution : class_weight='balanced'

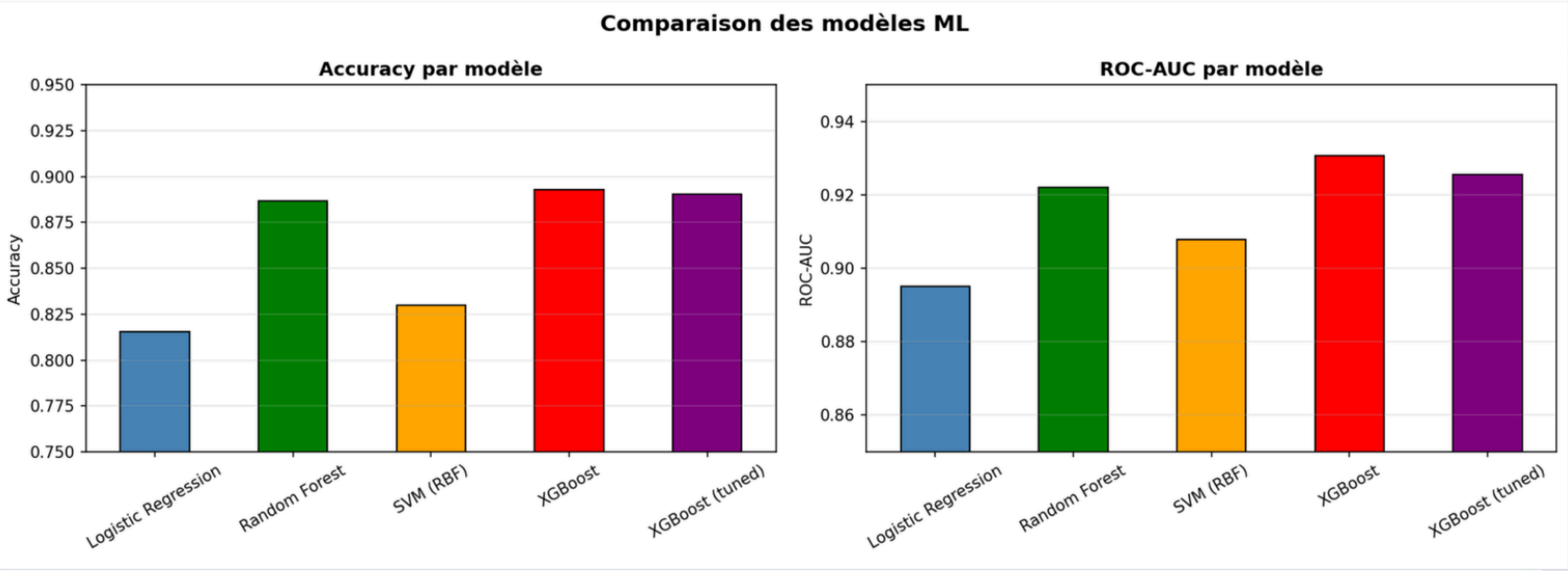
3. Modèles implémentés

- 1. Logistic Regression
  - → Modèle de référence (baseline)
  - → Frontière linéaire entre les classes
- 2. Random Forest
  - → 100 arbres de décision en parallèle
  - → Chaque arbre vote, la majorité gagne
- 3. SVM (kernel RBF)
  - → Cherche la meilleure frontière non-linéaire
  - → Entraîné sur 15 000 lignes (contrainte performance)
- 4. XGBoost (Gradient Boosting)
  - → Arbres construits séquentiellement
  - → Chaque arbre corrige les erreurs du précédent

4. Résultats — Comparaison des modèles

Modèle	Accuracy	ROC-AUC	Faux Négatifs
---	---	---	---
Logistic Regression	0.815	0.895	11.5%
Random Forest	0.887	0.922	32.0%
SVM (RBF)	0.830	0.908	14.1%
XGBoost	0.893	0.931	25.9%
XGBoost (tuned)	0.890	0.926	28.0%

4. Résultats — Comparaison des modèles



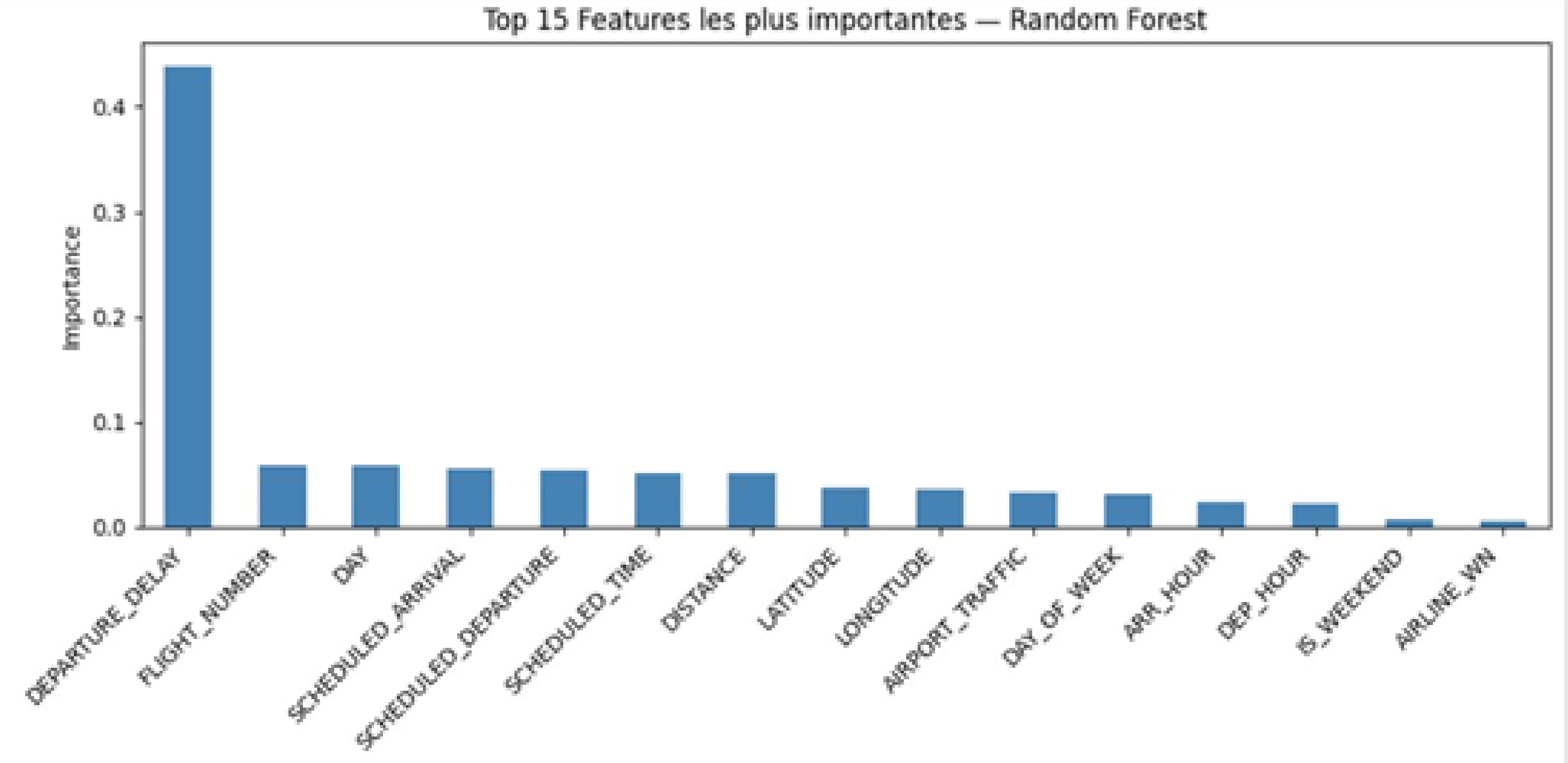
5. Quelles variables influencent le plus les retards ?

- **- DEPARTURE_DELAY représente 44% de l'importance totale**
  → Un retard au départ annonce fortement un retard à l'arrivée

- **- Les autres variables ont une importance équilibrée (~5% chacune) :**
  → FLIGHT_NUMBER, DAY, SCHEDULED_ARRIVAL, DISTANCE...

- **- Les compagnies aériennes ont très peu d'impact**
  → AIRLINE_WN < 1%

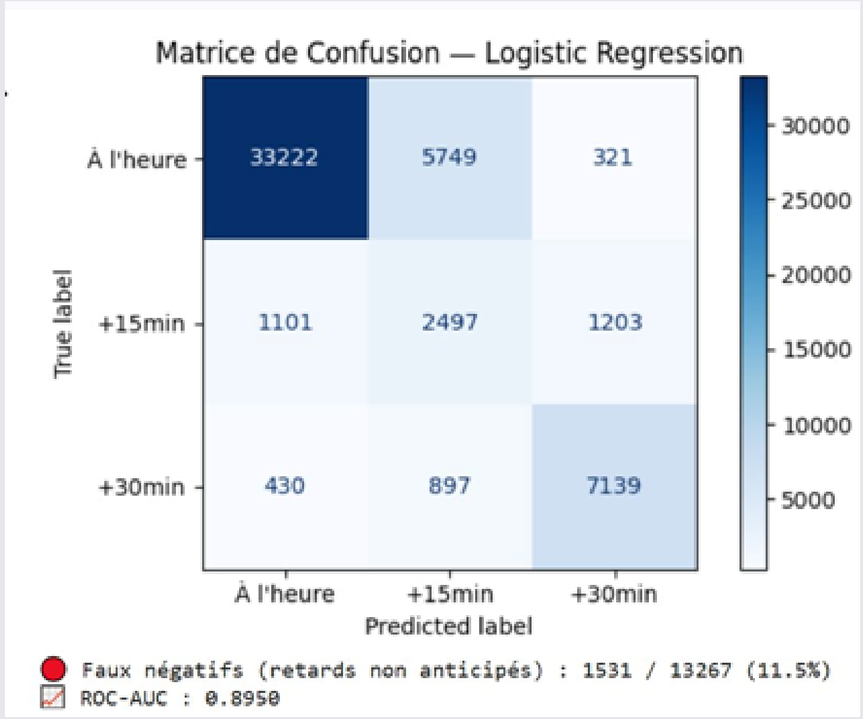
5. Quelles variables influencent le plus les retards ?

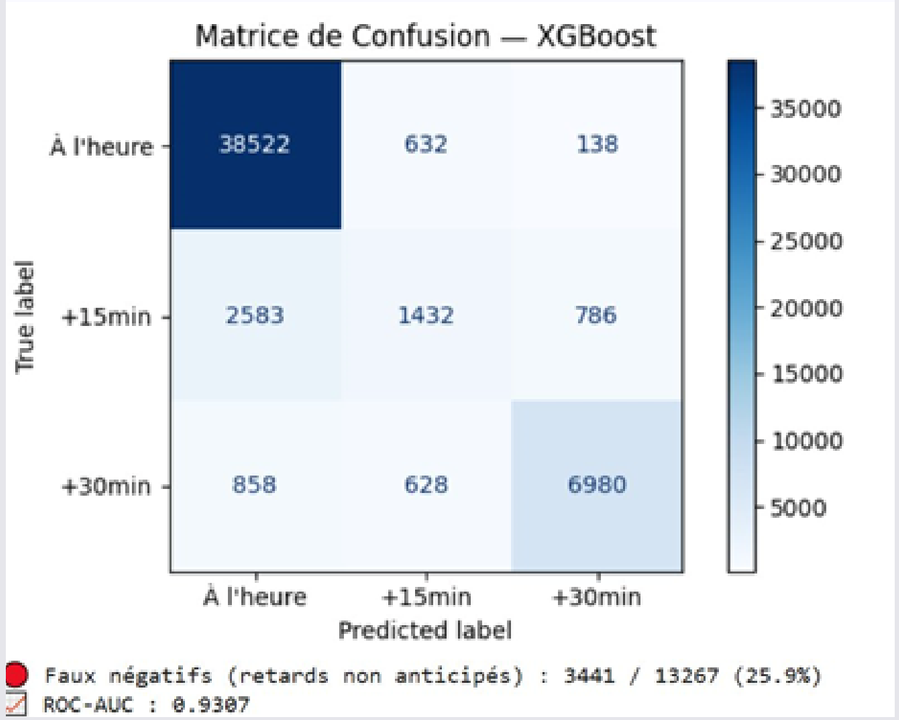


6. Analyse des prédictions — Matrices de Confusion

- **- Diagonale = bonnes prédictions**
- **- La classe +15 min est la plus difficile à prédire**
  → Peu de données (9% seulement)

6. Analyse des prédictions — Matrices de Confusion





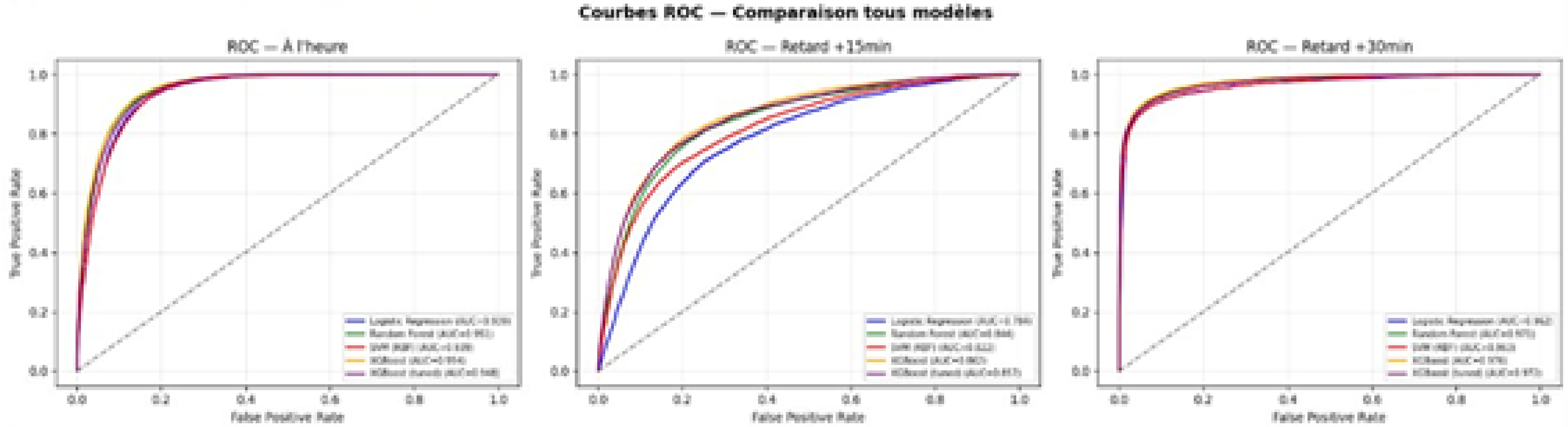
LR : 81,5% accuracy - 0,8950 ROC AUC=> frontière insuffisante
RF :Accuracy amelioré 88,6% ,
SVM : bien sur les détéctions des retars
XGBoost : meilleure 89,3% accuracy , 0,9508 ROC AUC => domine sur 3 classes (+15 min diff)
On démarche progressive sur la complexité XGBoost meilleur modèle

7. Courbes ROC — Performance par classe

- **Utilisation importante**
- **évaluer chaque modèle indépendement de seuil de décision**
- **- Plus la courbe est en haut à gauche → meilleur le modèle**
- **- XGBoost domine sur les 3 classes :**
  → Classe À l'heure : AUC = 0.954
  → Classe +15 min : AUC = 0.862
  → Classe +30 min : AUC = 0.976

7. Courbes ROC — Performance par classe

⏳ Génération des courbes ROC...



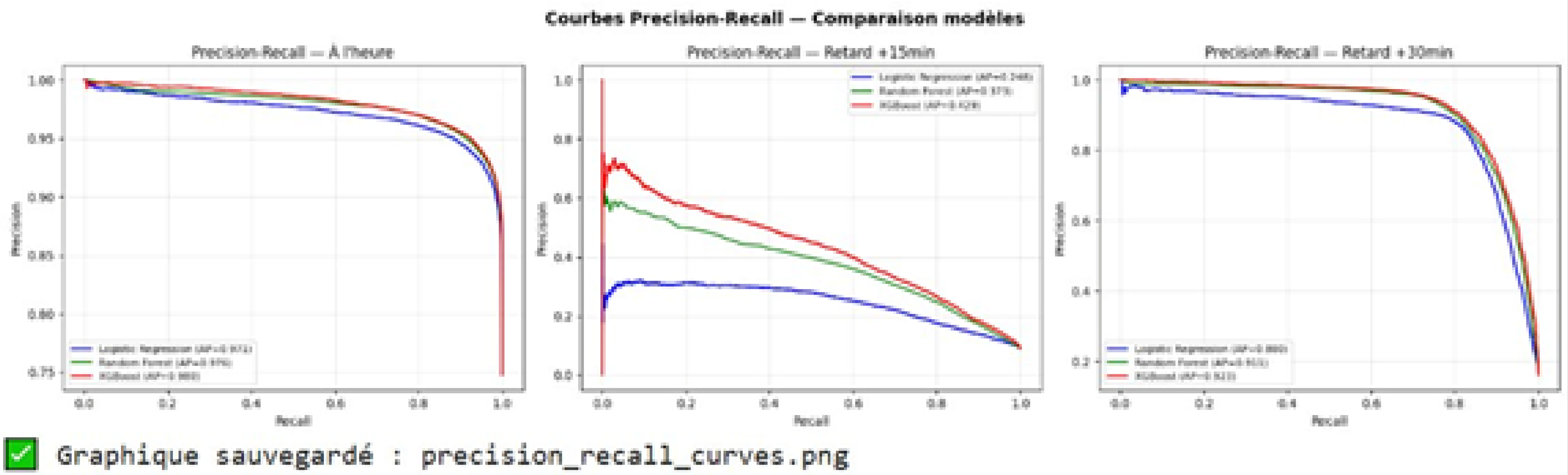
✅ Graphique sauvegardé : roc_curves.png

Pricipe : plus courbe est haut a gauche => meilleure modèle
Sur 3 classes XGboost domine clairement 0,952 - 0,974 - 0,846 (LR 0,774)
Résultat confirme que XGBoost capture beaucoup mieux la complixités de données

8. Courbes Precision-Recall

- **- Métrique clé quand les classes sont déséquilibrées**
- **- XGBoost meilleur sur la classe critique +15 min**
  - **→ Average Precision = 0.429**
  - **→ vs Logistic Regression : 0.248**

8. Courbes Precision-Recall



l'accuracy peut etre trompeuse quand une classe miniritaire commen +15 min
Sur cette classe critique XGBoost obtient avrege de précision de 0,419presque double de LR ce qui cofirme la performace de XGBoost

9. Optimisation — GridSearchCV

- - 16 combinaisons de paramètres testées
- - Validation croisée 3 folds = 48 entraînements au total
- Paramètres testés :
  → n_estimators : [100, 200]
  → max_depth : [4, 6]
  → learning_rate : [0.05, 0.1]
  → subsample : [0.8, 1.0]
- Meilleurs paramètres :
- learning_rate=0.1 | max_depth=4 | n_estimators=200 | subsample=0.8
- ⚠️ Observation : le tuning n'améliore pas les performances

→ XGBoost était déjà bien configuré par défaut

10. Analyse des Faux Négatifs ⚠️

- **Faux négatif = vol retardé prédit "à l'heure"**
- **C'est la métrique la plus critique du projet**
  → Un passager non averti = mauvaise expérience
- **Comparaison :**
  → Logistic Regression : 11.5% ✅ (le meilleur)
  → SVM : 14.1%
  → XGBoost : 25.9%
  → Random Forest : 32.0% ❌
- **Trade-off important :**
  → LR détecte mieux les retards MAIS accuracy globale faible
  → XGBoost meilleur globalement MAIS rate plus de retards

Partie 3:
Deep learning

Plan

01 **Préparation des Données**

Chargement, split 80/10/10, class weights

02 **MLP avec Optuna**

Architecture 3 couches + recherche bayésienne

03 **LSTM & GRU**

Modèles récurrents pour séries temporelles

04 **Transformer**

Attention multi-tête, encodage positionnel

05 **Analyses complémentaires**

Batch size, SHAP, tuning Optuna

06 **Comparaison & Conclusion**

Résultats, recommandations

1.Préparation de données

Pipeline de données

📁 **50 000 lignes chargées**

flights_cleaned_encoded.csv

🚫 **Exclusion DEPARTURE_DELAY**

Éviter le data leakage

⚖️ **Standardisation (StandardScaler)**

Moyenne 0, écart-type 1

✂️ **Split stratifié 80/10/10**

Train / Validation / Test

📦 **DataLoaders PyTorch**

batch_size = 256 par défaut

```
Distribution DELAY_CLASS :
DELAY_CLASS
0    0.693
2    0.192
1    0.115
```

02 · MLP + Optuna

Modèle 1 — MLP avec Recherche Bayésienne (Optuna)

Entrée	Couche 1	Couche 2	Couche 3	Sortie
N_FEATURES	256 → BN → ReLU → Drop	128 → BN → ReLU → Drop	64 → BN → ReLU → Drop/2	3 classes (Softmax)

Meilleurs Hyperparamètres (Optuna — 20 trials)

Hyperparamètre	Meilleure valeur
h1 — Couche 1	256 neurones
h2 — Couche 2	128 neurones
h3 — Couche 3	64 neurones
Dropout	0.30
Learning rate	0.00276

Résultats sur l'ensemble Test (5 000 vols)

Accuracy	**59 %**
ROC-AUC macro	**0.726**
F1 Retard +15 min	**0.22**
F1 Retard +30 min	**0.47**
Optimizer	**Adam + ReduceLROnPlateau**

Modèle 2 — LSTM et GRU (Séries Temporelles)

Chaque feature est traitée comme un pas de temps → tenseurs (batch, seq_len, 1) où seq_len = N_FEATURES

LSTM

Paramètre	Valeur
hidden_size	64
num_layers	2
Dropout	0.3
Gradient clipping	Oui
n_epochs	60
Learning rate	5e-4

Accuracy : **58 %** ROC-AUC : **0.707**

GRU

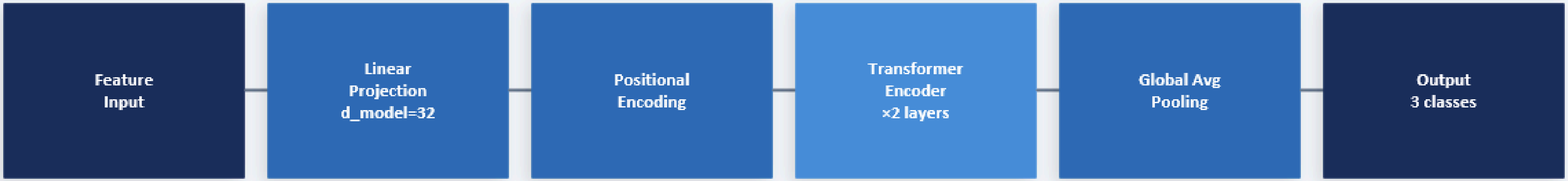
Paramètre	Valeur
hidden_size	64
num_layers	2
Dropout	0.3
Gradient clipping	Oui
n_epochs	60
Learning rate	5e-4

Accuracy : **59 %** ROC-AUC : **0.711**

04 · Transformer

Modèle 3 — Transformer Time Series

Chaque feature → token de dimension d_model · Encodage positionnel appris · Classification par Global Average Pooling

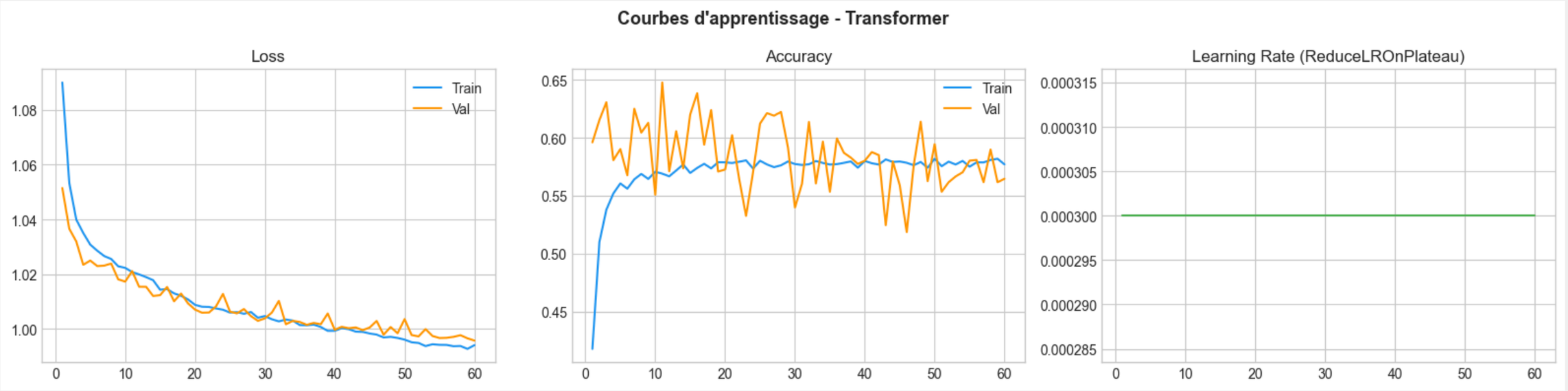


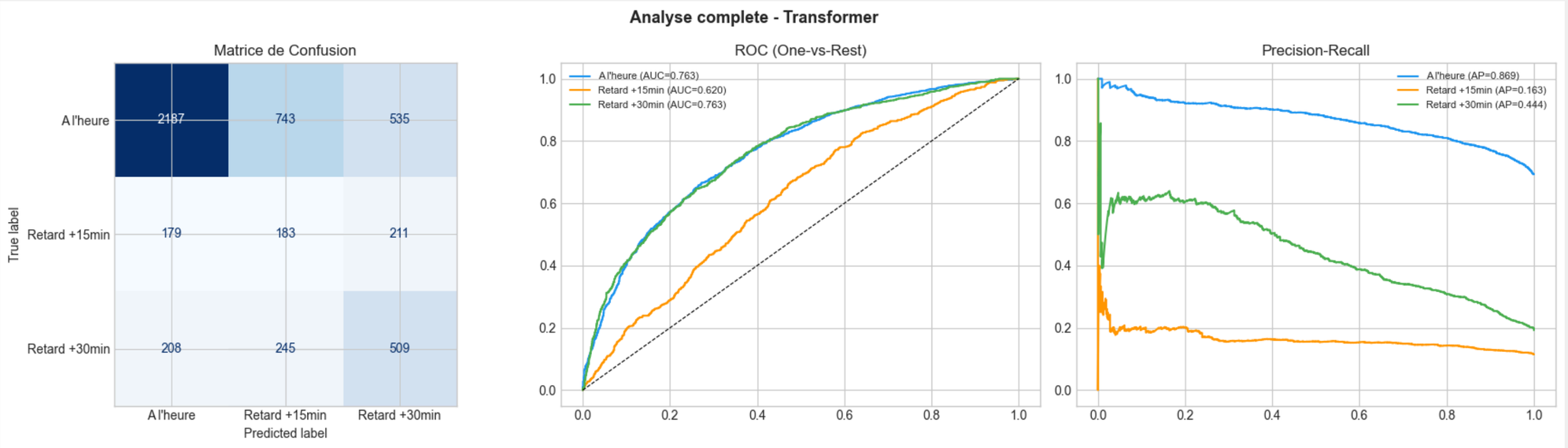
Paramètres d'architecture

Paramètre	Valeur
d_model	32
nhead	4
num_layers	2
dim_feedforward	128 (d_model × 4)
Dropout	0.2
Learning rate	3e-4

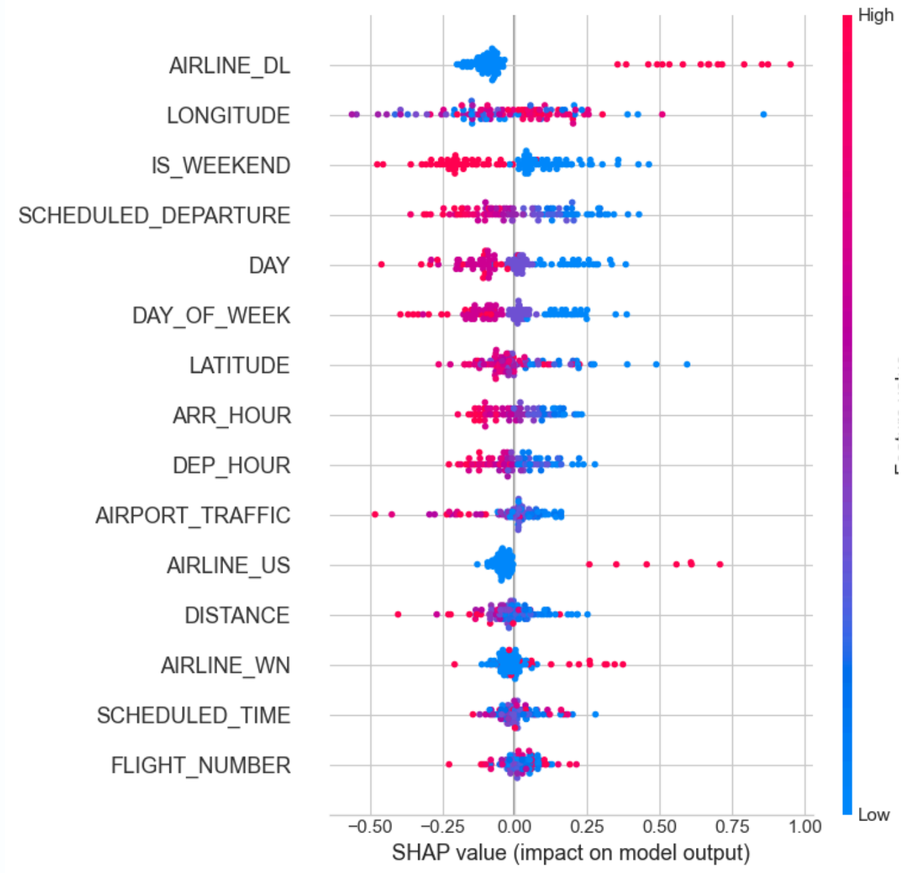
Résultats & Scores

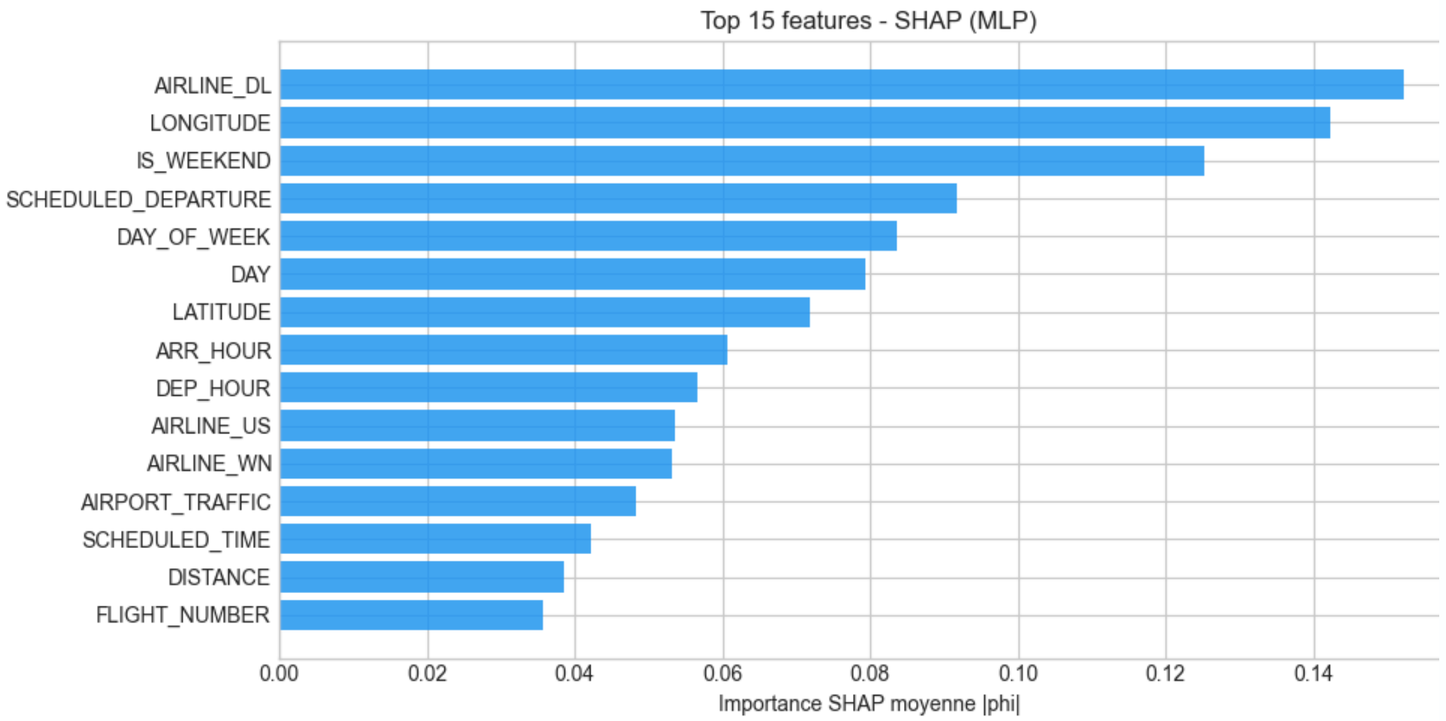
Accuracy	**60 %**
ROC-AUC macro	**0.715**
F1 Retard +15 min	**0.20**
F1 Retard +30 min	**0.46**


Courbes d'apprentissage - Transformer
Loss
Train
Val
Accuracy
Train
Val
Learning Rate (ReduceLROnPlateau)


Analyse complete - Transformer
Matrice de Confusion
A l'heure
Retard +15min
Retard +30min
True label
Predicted label
2187
743
535
179
183
211
208
245
509
ROC (One-vs-Rest)
A l'heure (AUC=0.763)
Retard +15min (AUC=0.620)
Retard +30min (AUC=0.763)
Precision-Recall
A l'heure (AP=0.869)
Retard +15min (AP=0.163)
Retard +30min (AP=0.444)

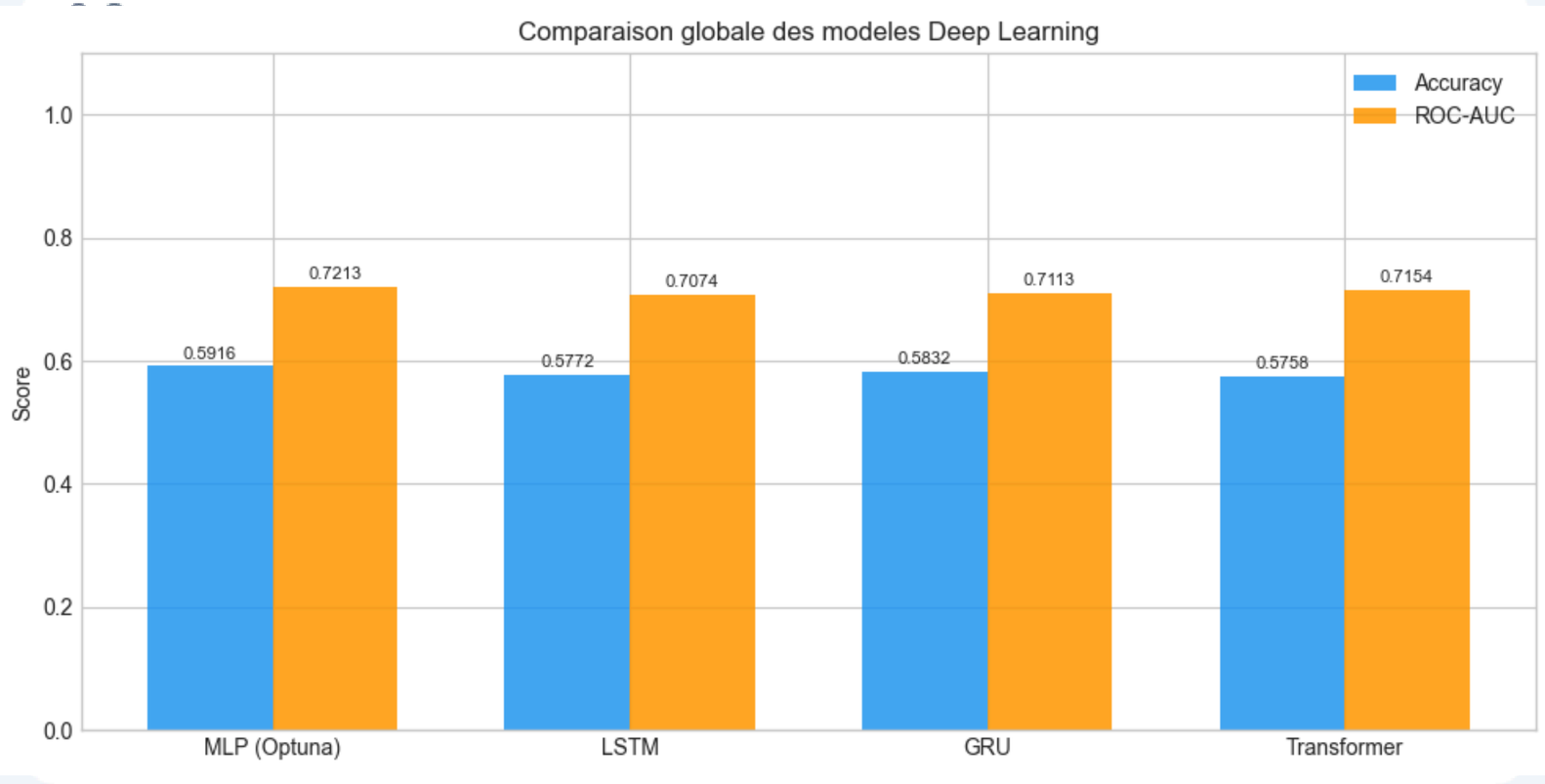
étude de la taille de batch et importance SHAP





06 · Comparaison

Comparaison Globale — Modèles Deep Learning



Synthèse

Modèle	Acc.	AUC
MLP (Optuna)	58.6%	0.726
LSTM	57.7%	0.707
GRU	58.7%	0.711
Transformer	60.1%	0.715

MLP
✅ Rapide · interprétable · bon équilibre
⚠ Pas de mémoire séquentielle

LSTM
✅ Dépendances temporelles longues
⚠ Lent · vanishing gradient

GRU
✅ Plus léger que LSTM · convergence rapide
⚠ Moins expressif sur longues séquences

Transformer
✅ Attention globale · scalable
⚠ Moins efficace sur petit dataset

Conclusion & Synthese

Resultats des modeles DL

Modele	Atouts	Limites
MLP (Optuna)	Rapide, interpretable, bon equilibre	Pas de memoire sequentielle
LSTM	Capture dependances temporelles	Lent, vanishing gradient possible
GRU	Plus leger que LSTM	Moins expressif sur longues sequences
Transformer	Attention globale, scalable	Moins efficace sur petit dataset

Recommandation

Le **MLP tune par Optuna** est le meilleur compromis performance/temps de calcul. Le **GRU** est recommande si les donnees sont organisees en sequences chronologiques. Les **Faux Negatifs** (vols retardes non detectes) representent le risque metier le plus important et doivent etre minimises via le recall sur les classes 1 et 2.